Student Name: Bhavya **UID: 24BAI70791**
Branch: CSE(AI & ML) **Section/Group: 24AIT_KRG-1/G2**
Semester: 4 **Date of Performance: 27/02/2026**
Subject Name: Database Management System **Subject Code: 24CSH-298**

Experiment – 6

1. Aim of the Session

To understand the concept and working of **cursors** in PL/SQL for **row-by-row data processing**, and to analyze how **implicit cursors**, **explicit cursors**, and **cursor attributes** are used to implement business logic on multiple rows in a database table.

2. Software Requirements

Database Management System:

- Oracle Database Express Edition (Oracle XE)
- PostgreSQL Database

Database Administration Tool / Client Tool:

- Oracle SQL Developer (for Oracle XE)
- pgAdmin (for PostgreSQL)

3. Objectives

To implement and analyze the use of **implicit cursors**, **explicit cursors**, and **cursor attributes** for processing multiple rows from a database table and applying business logic effectively.

4. Procedure of the Experiment

1. Create an **employee table** with fields **EmpID, EmpName, and Salary**.
2. Insert **sample employee records** into the table using the **INSERT command**.
3. Display all employee records from the table using the **SELECT statement**.
4. Write a **PL/SQL block using an implicit cursor** to perform operations such as **UPDATE or DELETE** on the employee table.
5. Use the **SQL%ROWCOUNT cursor attribute** to check how many rows were affected by the operation.
6. Declare an **explicit cursor** to retrieve employee records from the table.
7. **Open the cursor** using the **OPEN statement** to begin processing the records.
8. **Fetch records one by one** from the cursor using the **FETCH statement**.
9. Use a **LOOP structure** to process each employee record individually.
10. Apply **cursor attributes** such as **%FOUND, %NOTFOUND, %ROWCOUNT, and %ISOPEN** to control the program flow.
11. Display the processed employee details using **DBMS_OUTPUT.PUT_LINE**.
12. **Close the cursor** using the **CLOSE statement** after all records are processed.
13. Execute the program and **observe the output**.

5. Practical / Experiment Steps

PROGRAM CODE:

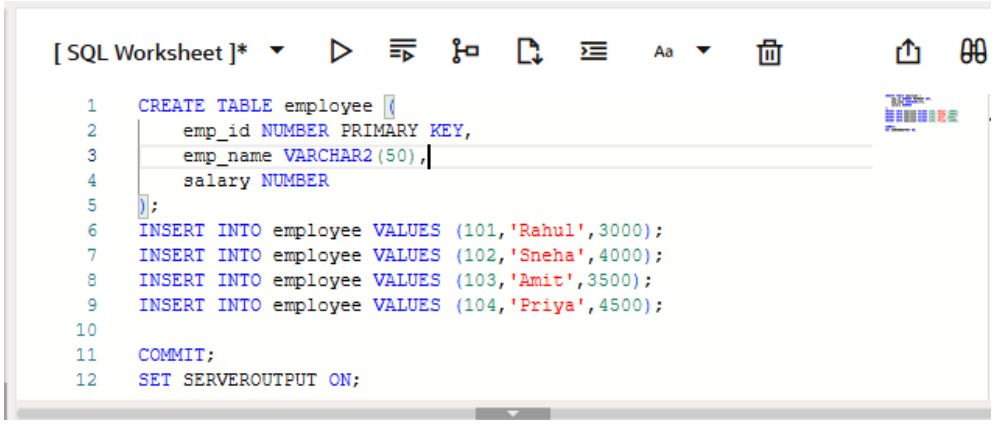
```
CREATE TABLE employee (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(50),
    salary NUMBER
);
```

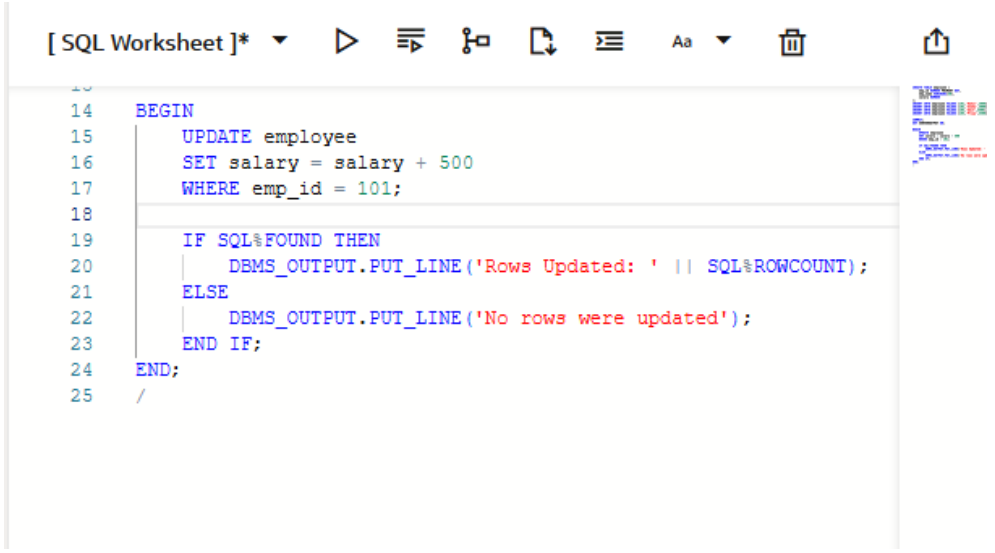

```
INSERT INTO employee VALUES (101,'Rahul',3000);

INSERT INTO employee VALUES (102,'Sneha',4000);

INSERT INTO employee VALUES (103,'Amit',3500);

INSERT INTO employee VALUES (104,'Priya',4500);

COMMIT;

SET SERVEROUTPUT ON;

-----------------------------------------------

-- Problem 1: Implicit Cursor

-----------------------------------------------

BEGIN

   UPDATE employee

   SET salary = salary + 500

   WHERE emp_id = 101;

   IF SQL%FOUND THEN

      DBMS_OUTPUT.PUT_LINE('Rows Updated: ' || SQL%ROWCOUNT);

   ELSE

      DBMS_OUTPUT.PUT_LINE('No rows were updated');

   END IF;

END;

/

-----------------------------------------------

-- Problem 2: Explicit Cursor

-----------------------------------------------

DECLARE

   CURSOR emp_cursor IS

      SELECT emp_id, emp_name, salary FROM employee;
```


```
    v_id employee.emp_id%TYPE;
    v_name employee.emp_name%TYPE;
    v_salary employee.salary%TYPE;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO v_id, v_name, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_name || ' ' || v_salary);
    END LOOP;
    CLOSE emp_cursor;
END;
/

-----------------------------------------------
-- Problem 3: Cursor Attributes
-----------------------------------------------
DECLARE
    CURSOR emp_cursor IS
        SELECT emp_id FROM employee;
    v_id employee.emp_id%TYPE;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO v_id;
        IF emp_cursor%FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_id);
```

```
    END IF;
    EXIT WHEN emp_cursor%NOTFOUND;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Total Rows Fetched: ' || emp_cursor%ROWCOUNT);
  CLOSE emp_cursor;
END;
/
```

6. Input / Output Details and Screenshot





Query result | Script output | DBMS output | Explain Plan | SQL history

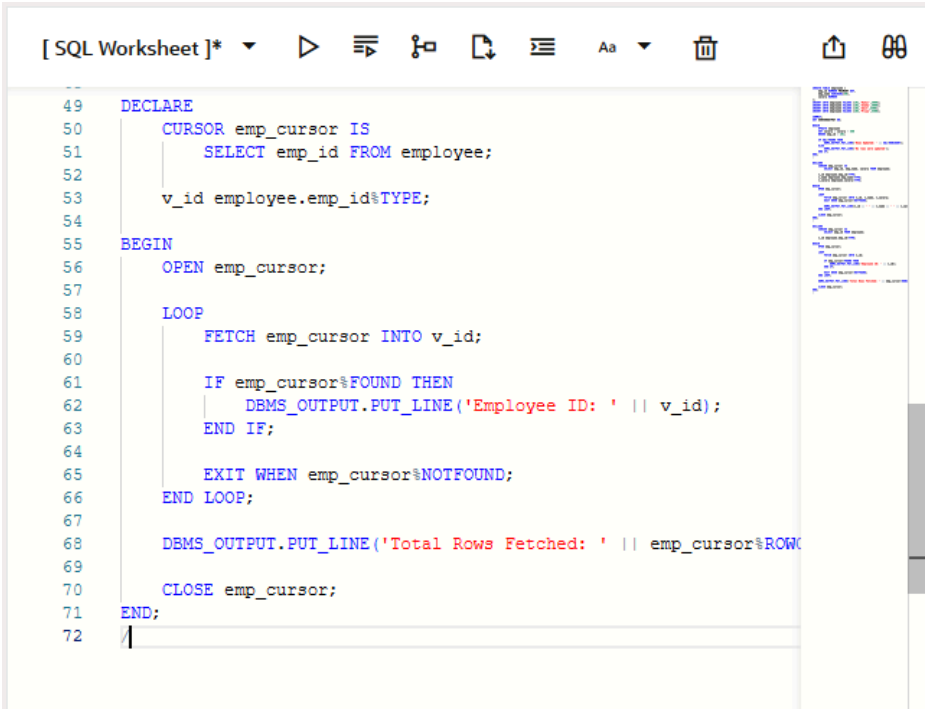
```
Rows Updated: 1


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.009
```

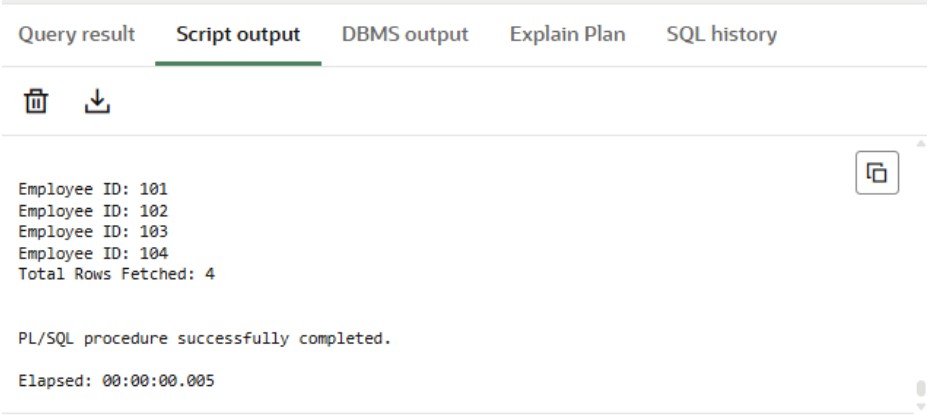
[SQL Worksheet]*

```
26
27  DECLARE
28      CURSOR emp_cursor IS
29          SELECT emp_id, emp_name, salary FROM employee;
30
31      v_id employee.emp_id%TYPE;
32      v_name employee.emp_name%TYPE;
33      v_salary employee.salary%TYPE;
34
35  BEGIN
36      OPEN emp_cursor;
37
38      LOOP
39          FETCH emp_cursor INTO v_id, v_name, v_salary;
40          EXIT WHEN emp_cursor%NOTFOUND;
41
42          DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_name || ' ' || v_sal
43      END LOOP;
44
45      CLOSE emp_cursor;
46  END;
47  /
```

Query result | Script output | DBMS output | Explain Plan | SQL history

```
101 Rahul 4000
102 Sneha 4000
103 Amit 3500
104 Priya 4500


PL/SQL procedure successfully completed.

Elapsed: 00:00:00.105
```





7. Learning Outcome

After completing this experiment, we learned:

- The concept and importance of **cursors in PL/SQL**
- The difference between **implicit and explicit cursors**
- How to use **cursor attributes** such as `%FOUND`, `%NOTFOUND`, `%ROWCOUNT`, and `%ISOPEN`
- How to process database records **row by row**
- Practical implementation of cursors in database programs